

第 3 讲

分 治

主要内容



3.1 引言

3.2 分治法策略的算法设计思想及分析方法

3.3. 分治策略的经典应用（可选讲部分）

主要内容

3.1 引言

3.1.1 简单问题例的算法描述

3.1.2 简单问题例算法的分析



3.1.1 简单问题例的算法描述——二分查找

? 问题描述

- 在数组 $A[1..n]$ 中搜索已知元素 x

• 基本思想

将一个给定的元素 x 与一个已排序数组 $A[\text{low}..\text{high}]$ 的中间元素做比较

- 如果 $x < A[\text{mid}]$, 这里 $\text{mid} = \lfloor (\text{low} + \text{high})/2 \rfloor$, 则不考虑 $A[\text{mid}..\text{high}]$, 而对 $A[\text{low}..\text{mid}-1]$ 重复实施相同的方法。
- 类似地, 如果 $x > A[\text{mid}]$, 则放弃 $A[\text{low}..\text{mid}]$, 而对 $A[\text{mid}+1..\text{high}]$ 重复实施相同的方法。

3.1.1 简单问题例的算法描述—二分查找伪代码

算法 3.1.1 BINARYSEARCHREC

输入：按非降序排列的 n 个元素的数组 $A[1..n]$ 和元素 x 。

输出：如果 $x=A[j]$ ， $1 \leq j \leq n$ ，则输出 j ；否则输出 0

1. *binarysearch*(1, n)

过程 *binarysearch*($low, high$)

1. **if** $low > high$ **then return** 0

2. **else**

3. $mid \leftarrow \lfloor (low+high)/2 \rfloor$

4. **if** $x=A[mid]$ **then return** mid

5. **else if** $x < A[mid]$ **then return** *binarysearch*($low, mid-1$)

6. **else return** *binarysearch*($mid+1, high$)

7. **end if**

3.1.1 简单问题例的算法描述—二分查找算法分析



假设

假定每个三向比较当做一次比较来计算

$$C(n) \leq \begin{cases} 1 & \text{若 } n = 1 \\ 1 + C(\lfloor n/2 \rfloor) & \text{若 } n \geq 2 \end{cases}$$



定理 6.2

算法 BINARYSEARCHREC 在 n 个元素组成的数组中搜索某个元素所执行的元素比较次数不超过 $\lceil \log n \rceil + 1$ 。算法 BINARYSEARCHREC 的时间复杂性是 $O(\log n)$ 。

算法递归深度为 $O(\log n)$ ，并且由于每个递归层次需要 $\lceil \log n \rceil$ 的空间，本算法所需的空间总量是 $O(\log n)$ 。和递归算法相反，其迭代形式算法仅需要 $\lceil \log n \rceil$ 空间



3.1.1 简单问题例的算法描述——归并排序

- BOTTOMUPSORT :

- 在每一层，我们有已排序的序列对，同时它们两两被合并而得到较大的排序序列。
- 沿着树一层一层向上继续这个过程，直到到达根为止，这最后的序列是已经排序好的。

? 让我们从反向来考虑，是否能用自顶向下取代自底向上？

3.1.1 简单问题例的算法描述—归并排序基本思想



基本思想

- 首先，将该数组分成两个各一半元素的数组

$A[1 \cdots 8] =$

9	4	5	2	1	7	4	6
---	---	---	---	---	---	---	---



9	4	5	2
---	---	---	---

 和

1	7	4	6
---	---	---	---

- 接着分别对这两个数组的元素排序
- 然后只是将它们合并来得到所希望的排序数组
- 对每一半子数组，可随意使用任何排序算法来对这两个子数组排序。特别是可以使用算法 SORT 本身。
- 得出了著名的算法 MERGESORT，此算法的完整描述见下面。

3.1.1 简单问题例的算法描述—归并排序伪代码

算法 3.1.2 Mergesort

输入：n 个元素的数组 $A[1..n]$ 。

输出：按非降序排列的数组 $A[1..n]$ 。

1. *mergesort*($A, 1, n$)

过程 *mergesort*($A, low, high$)

1. **if** $low < high$ **then**

2. $mid \leftarrow \lfloor (low + high) / 2 \rfloor$

3. *mergesort*(A, low, mid)

4. *mergesort*($A, mid + 1, high$)

5. *MERGE*($A, low, mid, high$)

6. **end if**

3.1.1 简单问题例的算法描述—归并排序例子

- 算法 MERGESORT 对下列数组 A 排序的操作过程：
 $A = \{9, 4, 5, 2, 1, 7, 4, 6\}$.
- 定义
 - mergesort(A, low, high) 为 MS(low, high).
 - Merge(A, low, mid, high) 为 M(low, mid, high).

3.1.1 简单问题例的算法描述—归并排序例子

$MS(1,8)$

$M(1,4,8)$

9	4	5	2	1	7	4	6
1	2	4	4	5	6	7	9

$MS(1,4)$

$M(1,2,4)$

9	4	5	2
2	4	5	9

$MS(5,8)$

$M(5,6,8)$

1	7	4	6
1	4	6	7

$MS(1,2)$

$M(1,1,2)$

9	4
4	9

$MS(3,4)$

$M(3,3,4)$

5	2
2	5

$MS(5,6)$

$M(5,5,6)$

1	7
1	7

4	6
4	6

$MS(7,8)$

$M(7,7,8)$

$MS(1,1)$

$MS(2,2)$

$MS(3,3)$

$MS(4,4)$

$MS(5,5)$

$MS(6,6)$

$MS(7,7)$

$MS(8,8)$

$M(1,1,1)$

$M(2,2,2)$

$M(3,3,3)$

$M(4,4,4)$

$M(5,5,5)$

$M(6,6,6)$

$M(7,7,7)$

$M(8,8,8)$

9
9

4
4

5
5

2
2

1
1

7
7

4
4

6
6

Success!

3.1.1 简单问题例的算法描述——归并排序操作过程

- 这个调用过程对应于树的前序遍历：访问根、左子树和右子树。
- 计算过程却对应于树的后序遍历：访问左子树，右子树，最后是根。
- 为了实现这个遍历，用一个栈来存储每次调用过程的局部数据。
- 在算法 BOTPOMUPSORT 中，合并是一层接一层进行的；而在算法 MERGESORT 中，合并是以后序遍历的方式进行的。两种算法的区别仅在于合并的次序，因此，当 n 是 2 的幂时，由算法 MERGESORT 执行的比较次数和算法 BOTPOMUPSORT 执行的次数相同。

3.1.1 简单问题例的算法描述—归并排序算法分析

- 基本运算：元素比较。
- 为了简单，假定 n 是 2 的幂。
 - 如果 $n=1$ ，则算法不执行任何元素的比较，比较次数是 0。
 - 如果 $n>1$ ，则将问题分解为 (whose size is $n/2$) 2 个子问题 (whose size are $n/2$)。
 - 合并两个子数组所需的元素比较次数在 $n/2$ 与 $n-1$ 之间。这样算法所需的最小比较次数：

$$c(n) = \begin{cases} 0 & \text{if } n=1 \\ 2c(n/2) + n/2 & \text{if } n > 1 \end{cases} \quad \longrightarrow \quad c(n) = \frac{n \log n}{2}$$

3.1.1 简单问题例的算法描述—归并排序算法分析

- 最大比较次数:

$$c(n) = \begin{cases} 0 & \text{if } n = 1 \\ 2c(n/2) + n - 1 & \text{if } n \geq 2 \end{cases}$$

$$c(n) = 2c(n/2) + n - 1$$

$$= 2(2c(n/2^2) + n/2 - 1) + n - 1$$

$$= 2^2 c(n/2^2) + n - 2 + n - 1$$

$$= 2^2 c(n/2^2) + 2n - 2 - 1$$

.....

$$= 2^k c(n/2^k) + kn - 2^{k-1} - 2^{k-2} - \dots - 2 - 1$$

$$= 2^k c(1) + kn - \sum_{j=0}^{k-1} 2^j$$

$$= 2^k \times 0 + kn - (2^k - 1)$$

$$= kn - 2^k + 1$$

$$= n \log n - n + 1$$

3.1.1 简单问题例的算法描述—归并排序算法分析

- 如果 n 是任意的正整数 (不必是 2 的幂), 对于由算法 MERGESORT 执行的元素比较次数:

$$c(n) = \begin{cases} 0 & \text{if } n = 1 \\ c(\lfloor n/2 \rfloor) + c(\lceil n/2 \rceil) + bn & \text{if } n \geq 2 \end{cases}$$

$$c(n) = \Theta(n \log n)$$



定理 6.3 算法 MERGESORT 对一个 n 个元素的数组排序所需的时间是 $\Theta(n \log n)$, 空间是 $\Theta(n)$ 。



3.1.2 简单问题例算法的分析

- 对于一个规模为 n 的问题：若该问题可以容易地解决（比如说规模 n 较小）则直接解决，否则将其分解为 k 个规模较小的子问题，这些子问题互相独立且与原问题形式相同，递归地解这些子问题，然后将各子问题的解合并得到原问题的解。
- 这种算法设计策略叫做**分治法**。

主要内容



3.1 引言

3.2 分治法策略的算法设计思想及分析方法

3.3. 分治策略的经典应用（可选讲部分）

主要内容



3.2 分治法策略的算法设计思想及分析方法

3.2.1 分治算法的一般框架

3.2.2 分治算法的一般分析方法

3.2.3 分治策略和归纳策略的异同点



3.2.1 分治算法的一般框架—分治范式

分治范式由以下的步骤组成

- **划分步骤**。在算法的这个步骤中，输入被分成 $p \geq 1$ 个部分，每个子实例的规模严格小于 n ， n 是原始实例的规模。尽管比 2 大的其他小的常数很普遍，但 P 的最常用的值是 2。
- **治理步骤**。如果问题的规模大于某个预定的**阈值 n** ，这个步骤由执行 P 个递归调用组成，阈值是由算法的数学分析导出的
- **组合步骤**。这个步骤是组合 P 个递归调用的解来得到期望的输出值。

3.2.1 分治算法的一般框架—求解过程

分治法通常采用递归算法设计技术，在每一层递归上都有 3 个步骤：

- ① **分解**：将原问题分解为若干个规模较小，相互独立，与原问题形式相同的子问题。
- ② **求解子问题**：若子问题规模较小而容易被解决则直接求解，否则递归地求解各个子问题。
- ③ **合并**：将各个子问题的解合并为原问题的解。

3.2.1 分治算法的一般框架—求解过程

```
divide-and-conquer(P)
```

```
{
```

```
    if (  $|P| \leq n_0$ ) adhoc(P); // 解决小规模的问题
```

```
    divide P into smaller subinstances  $P_1, P_2, \dots, P_k$  ; // 分解问题
```

```
    for (i=1, i $\leq$ k, i++)
```

```
         $y_i = \text{divide-and-conquer}(P_i)$ ; // 递归的解各子问题
```

```
    return merge( $y_1, \dots, y_k$ ); // 将各子问题的解合并为原问题的解
```

```
}
```



3.2.2 分治算法的一般分析方法—特征

分治法所能解决的问题一般具有以下几个特征：

- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的相同问题，即该问题具有最优子结构性质
- 利用该问题分解出的子问题的解可以合并为该问题的解；
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。



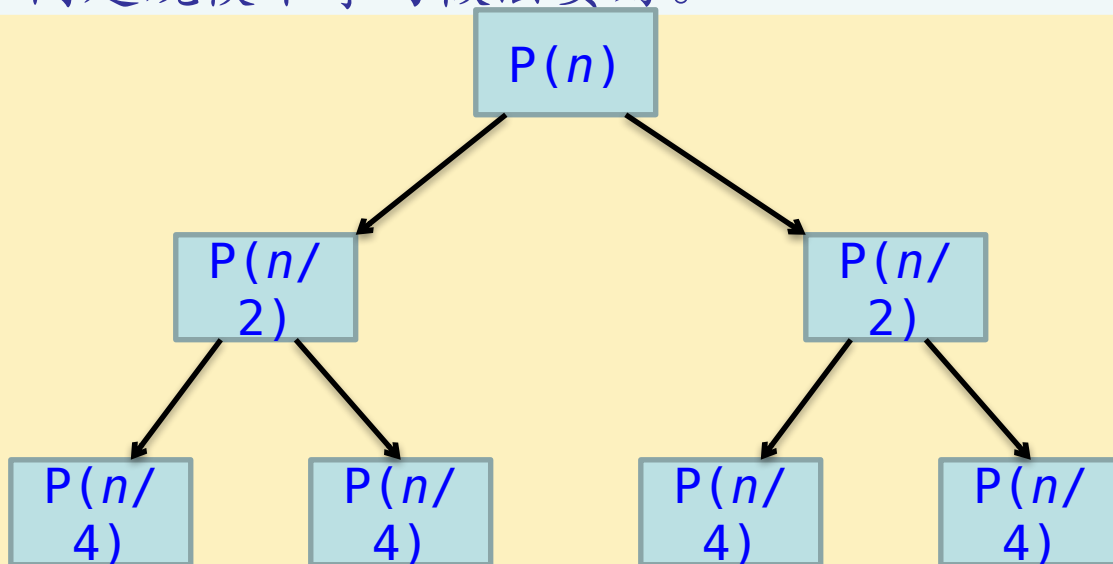
3.2.2 分治算法的一般分析方法—分割原则

- 根据分治法的分割原则，原问题应该分为多少个子问题才较适宜？各个子问题的规模应该怎样才为适当？
- 这些问题很难予以肯定的回答。但人们从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。换句话说，将一个问题分成大小相等的 k 个子问题的处理方法是行之有效的。
- 当 $k=1$ 时称为**减治法**。



3.2.2 分治算法的一般分析方法—分割原则

许多问题可以取 $k=2$ ，称为二分法，如下图所示，这种使子问题规模大致相等的做法是出自一种平衡子问题的思想，它几乎总是比子问题规模不等的做法要好。



...

...

...

...



3.2.3 分治策略和归纳策略的异同点

- 由分治法产生的子问题往往是原问题的较小模式，这就为使用递归技术提供了方便。在这种情况下，反复应用分治手段，可以使子问题与原问题类型一致而其规模却不断缩小，最终使子问题缩小到很容易直接求出其解。这自然导致递归过程的产生。
- 分治法与归纳法像一对孪生兄弟，经常同时应用在算法设计之中，并由此产生许多高效算法。

主要内容



3.1 引言

3.2 分治法策略的算法设计思想及分析方法

3.3. 分治策略的经典应用（可选讲部分）



主要内容



3.3. 分治策略的经典应用（可选讲部分）

3.3.1 寻找第 k 小元素

3.3.2 大整数乘法

3.3.3 矩阵乘法

3.3.4 最近点对

3.3.5 货郎担

3.3.6 多数元素

3.3.7 循环赛日程表



3.3.1 寻找第 k 小元素

- 由分治法产生的子问题往往是原问题的较小模式，这就为使用递归技术提供了方便。在这种情况下，反复应用分治手段，可以使子问题与原问题类型一致而其规模却不断缩小，最终使子问题缩小到很容易直接求出其解。这自然导致递归过程的产生。
- 分治法与归纳法像一对孪生兄弟，经常同时应用在算法设计之中，并由此产生许多高效算法。

3.3.1 寻找第 k 小元素—问题描述

? 选择问题

- 在一个包含 n 个元素的集合中寻找第 k 个最小元素。
(中值是第 $\lfloor n/2 \rfloor$ 个最小元素)

• 直接方法：

- 对所有的元素排序并取出中间一个元素，这个方法需要 $\Omega(n \log n)$ 时间，因为任何基于比较的排序过程在最坏的情况下必须至少要花费这么多时间

? Can we find an efficient method in optimal linear time?

能否在最优线性时间内找到更快的算法?

3.3.1 寻找第 k 小元素— *Select* 算法

- 假设递归算法中，在每个递归调用的划分步骤后，我们丢弃元素的一个固定部分并且对剩余的元素递归。则问题的规模以几何级数递减，也就是在每个调用过程中，问题的规模以一个常因子被减小。
- 假设某个算法丢弃 1/3 并对剩余的 2/3 部分递归，那么在第二次调用中，元素的数目变为 $2n/3$ 个，第三次调用中为 $4n/9$ ，第四次调用中变为 $8n/27$ ，等等。
- 现在假定在每次调用中，算法对每个元素耗费的时间不超过一个常数，则耗费在处理所有元素上的全部时间产生一个几何级数：

$$cn + (2/3)cn + (2/3)^2 cn + \dots + (2/3)^j cn + \dots$$

这里。 c 是某个被适当选择的常数，由等式 2.12，这个总数小于

$$\sum_{j=0}^{\infty} cn(2/3)^j = 3cn = \Theta(n)$$

3.3.1 寻找第 k 小元素—例子



- 假设存在数组 $A[1..25]$:
 - 8, 33, 17, 51, 57, 49, 35, 11, 25, 37, 14, 3, 2, 13, 32, 12, 6, 29, 32, 54, 5, 16, 22, 23, 7
- 我们要在数组 A 中找到第 13 个最小的元素



3.3.1 寻找第 k 小元素—例子

8	33	17	51	57	49	35	11	25	37	14	3	2	13	52	12	6	29	32	54	5	16	22	23	7
---	----	----	----	----	----	----	----	----	----	----	---	---	----	----	----	---	----	----	----	---	----	----	----	---

partition

8	33	17	51	57	49	35	11	25	37	14	3	2	13	52	12	6	29	32	54	5	16	22	23	7
---	----	----	----	----	----	----	----	----	----	----	---	---	----	----	----	---	----	----	----	---	----	----	----	---

sort

8	17	33	51	57	11	25	35	37	49	2	3	13	14	52	6	12	29	32	54	5	7	16	22	23
---	----	----	----	----	----	----	----	----	----	---	---	----	----	----	---	----	----	----	----	---	---	----	----	----

median

13	16	29	33	35
----	----	----	----	----

partition

8	17	11	25	14	3	2	13	12	6	5	16	22	23	7	29	33	51	57	49	35	37	52	32	54
---	----	----	----	----	---	---	----	----	---	---	----	----	----	---	----	----	----	----	----	----	----	----	----	----

discard

3.3.1 寻找第 k 小元素—例子

A[1...15]

8	17	11	25	14	3	2	13	12	6	5	16	22	23	7
---	----	----	----	----	---	---	----	----	---	---	----	----	----	---

partition

8	17	11	25	14
---	----	----	----	----

3	2	13	12	6
---	---	----	----	---

5	16	22	23	7
---	----	----	----	---

sort

8	11	14	17	25
---	----	----	----	----

2	3	6	12	13
---	---	---	----	----

5	7	16	22	23
---	---	----	----	----

median

6	14	16
---	----	----

partition

8	11	3	2	13	12	6	5	7
---	----	---	---	----	----	---	---	---

14

17	25	16	22	23
----	----	----	----	----

discard

3.3.1 寻找第 k 小元素—例子

A[11...15]

17	25	16	22	23
----	----	----	----	----

sort

16	17	22	23	25
----	----	----	----	----

Succeed! The result is A[13]=22

3.3.1 寻找第 k 小元素—基本思想



基本思想

- 首先，如果元素个数小于预定义的**阈值** 44，则算法使用直接的方法计算第 k 小的元素，至于如何选择阈值在以后的算法分析中将会清楚。
- 下一步把 n 个元素**划分**成 $\lfloor n/5 \rfloor$ 组，每组由 5 个元素组成，如果 n 不是 5 的倍数，则排除剩余的元素，这应当不影响算法的执行。每组进行**排序**并取出它的中项即第三个元素
- 接着将这些中项序列中的中项元素记为 **mm**，它是通过递归计算得到的。将数组 A 中的元素划分成三个数组：A1, A2, A3，其中分别包含小于、等于和大于 mm 的元素。
- 最后，求出第 k 小的元素出现在三个数组中的哪一个，并根据测试结果，算法或者返回第 k 小的元素，或者在 A1 or A3 上递归。



3.3.1 寻找第 k 小元素—伪代码

算法 3.3.1 SELECT

输入: n 个元素的数组 $A[1..n]$ 和整数 $k, 1 \leq k \leq n$ 。

输出: A 中的第 k 小元素。

1. $\text{select}(A, 1, n, k)$

过程 $\text{select}(A, \text{low}, \text{high}, k)$

1. $p \leftarrow \text{high} - \text{low} + 1$

2. if $p < 44$ then 将 A 排序 return $(A[k])$

3. Let $q = \lfloor p/5 \rfloor$. 将 A 分成 q 组, 每组 5 个元素。则排除剩余的元素

4. 将 q 组中的每一组单独排序, 找出中项。所有

5. $mm \leftarrow \text{select}(M, 1, q, \lfloor q/2 \rfloor)$ { mm 为中项}

6. 将 $A[\text{low}..\text{high}]$ 分成三组:

$A1 = \{a | a < mm\}$ $A2 = \{a | a = mm\}$ $A3 = \{a | a > mm\}$

7. case

$|A1| \geq k$: return $\text{select}(A1, 1, |A1|, k)$

$|A1| + |A2| \geq k$: return mm

$|A1| + |A2| < k$: return $\text{select}(A3, 1, |A3|, k - |A1| - |A2|)$

8. end case

在算法中 select 过程的步骤 1 和步骤 2 耗费的时间都是 $\Theta(1)$

步骤 3 为 $\Theta(n)$ 时间。由于对每组排序需要时间为常量, 步骤 4 耗费 $\Theta(n)$ 时间,

步骤 5 所需的时间为 $T(\lfloor n/5 \rfloor)$

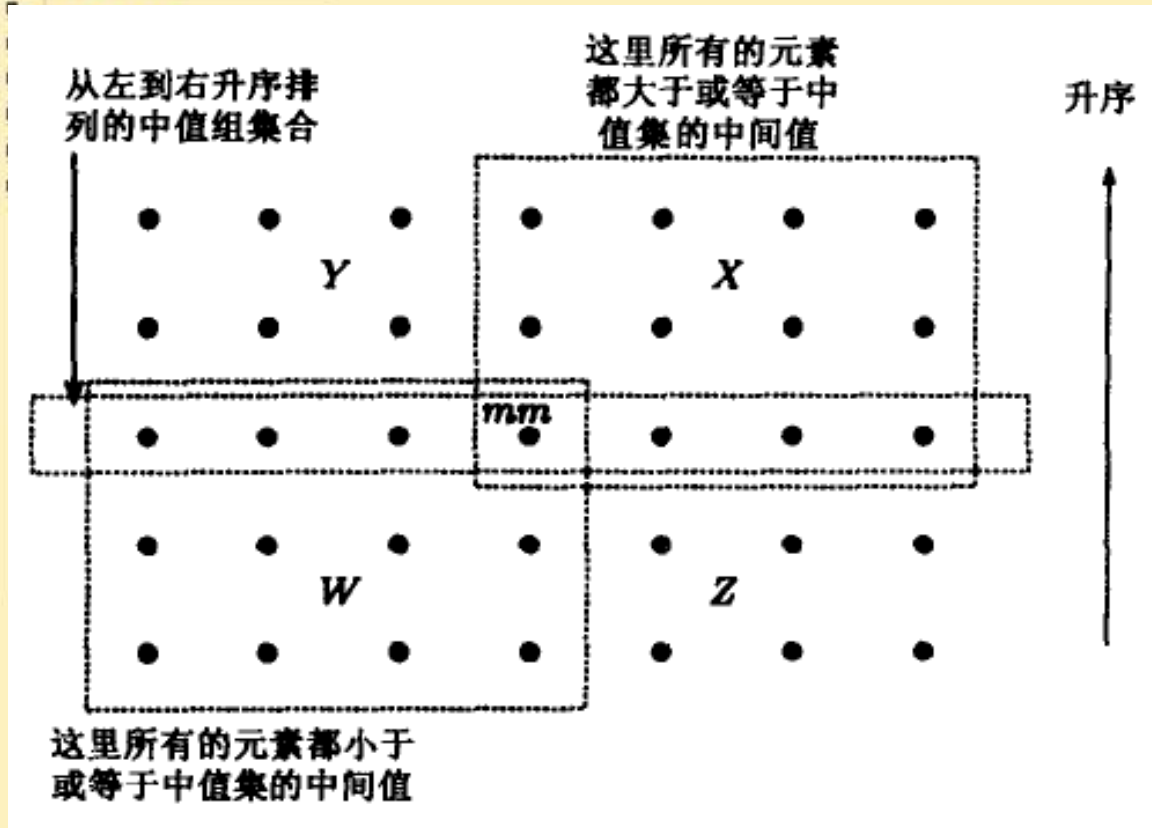
步骤 6 需要 $\Theta(n)$ 时间。由以上的分析,

步骤 7 耗费的时间至多是 $T(0.7n + 1.2)$ 。

当 $n \geq 44$ 时, 如果 $0.7n + 1.2 \leq 0.75n - 1$ 这个不等式将成立。得出的结论为, 对于 $n \geq 44$, 步骤 7 所耗费的时间至多是 $T(0.75n)$



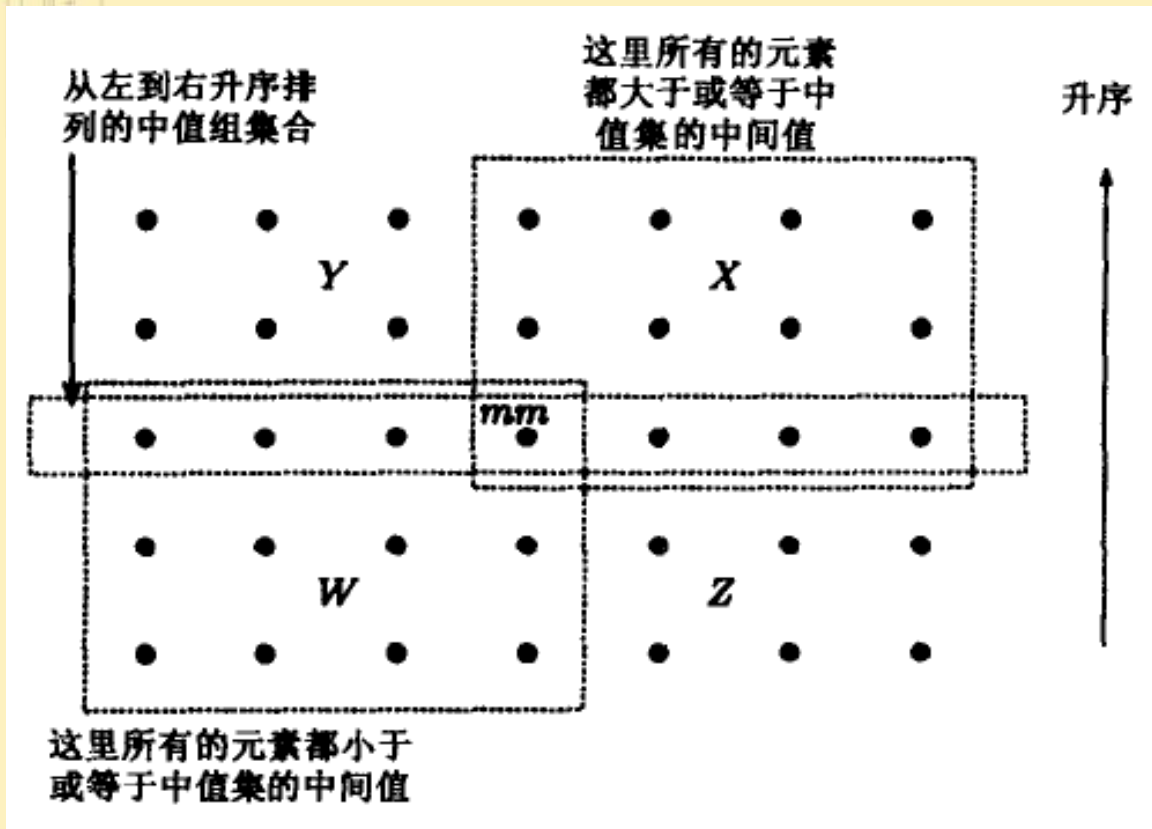
3.3.1 寻找第 k 小元素—算法分析



其中的元素已被划分成具有 5 元素的各个组，每组元素从底到顶按升序排列。

而且这些组以这样的方式整齐排列：它们的中项是从左到右以升序排列的。

3.3.1 寻找第 k 小元素—算法分析



所有围在矩形 W 中的元素是小于或等于 mm 的，所有围在矩形 X 中的元素是大于或等于 mm 的。

设 A'_1 表示小于或等于 mm 的元素集， A'_3 是大于或等于 mm 的元素集。这个算法中， A_1 是严格小于 mm 的元素集， A_3 是严格大于 mm 的元素集。

3.3.1 寻找第 k 小元素—算法分析

- 由于 A'_1 至少与 w 同样大, 我们有

$$|A'_1| \geq 3\lceil \lfloor n/5 \rfloor / 2 \rceil \geq \frac{3}{2}\lfloor n/5 \rfloor$$

- 因此 $|A_3| \leq n - \frac{3}{2}\lfloor n/5 \rfloor \leq n - \frac{3}{2}\left(\frac{n-4}{5}\right) = n - 0.3n + 1.2 = 0.7n + 1.2$.

- 由参数的对称性得到

$$|A'_3| \geq \frac{3}{2}\lfloor n/5 \rfloor \text{ and } |A_1| \leq 0.7n + 1.2$$

- 这样就为在 A_1 和 A_3 的元素个数建立了上界:
 - 即小于 mm 的元素个数和大于 mm 的元素个数不能够超过大约 $0.7n$ (n 的常分数倍)。

3.3.1 寻找第 k 小元素—算法分析

算法的运行时间 $T(n)$

- 在算法中 select 过程的步骤 1 和步骤 2 耗费的时间都是 $\Theta(1)$
- 步骤 3 为 $\Theta(n)$ 时间。由于对每组排序需要时间为常量，步骤 4 耗费 $\Theta(n)$ 时间，
- 步骤 5 所需的时间为 $T(\lfloor n/5 \rfloor)$ ，步骤 6 需要 $\Theta(n)$ 时间。由以上的分析，步骤 7 耗费的时间至多是 $T(0.7n+1.2)$ 。
- 当 $n \geq 44$ 时，如果 $0.7n + 1.2 \leq 0.75n - 1$ 这个不等式将成立。得出的结论为，对于 $n \geq 44$ ，步骤 7 所耗费的时间至多是 $T(\lfloor 0.75n \rfloor)$ 。

3.3.1 寻找第 k 小元素—算法分析

- 这个分析蕴含着算法 SELECT 的运行时间有以下的递推式

$$T(n) \leq \begin{cases} c & \text{if } n < 44 \\ T(\lfloor n/5 \rfloor) + T(\lfloor 3n/4 \rfloor) + cn & \text{if } n \geq 44 \end{cases}$$

$$T(n) \leq \frac{cn}{1 - 1/5 - 3/4} = 20cn$$

在一个由 n 个元素组成的线序集合中提取第 k 小元素，所需的时间是 $\Theta(n)$ ，特别地， n 个元素的中值可以在 $\Theta(n)$ 时间找出。

3.3.2 大整数乘法—问题描述

- 我们假定大小一定的整数相乘需要耗费定量的时间，而当两个任意长的整数相乘时，这个假定就不再有效了
- 对于处理不定长数的算法的数据输入，通常是按二进制的位数来计算的（或按二进制数字来计算）
- 设 u 和 v 是两个 n 位的整数，传统的乘法算法需要 $\Theta(n^2)$ 数字相乘来计算 u 和 v 的乘积。

? Is there much more efficient method?

3.3.2 大整数乘法—基本思想

- 设 u 和 v 都是 n 位的二进制整数，现在要计算它们的乘积 $u*v$ 。
- 我们可以用小学所学的方法来设计一个计算乘积 $u*v$ 的算法，但是这样做计算步骤太多，显得效率较低。如果将每 2 个 1 位数的乘法或加法看作一步运算，那么这种方法要作 $O(n^2)$ 步运算才能求出乘积 $u*v$ 。
- 下面我们用法来设计一个更有效的大整数乘积算法。
- 我们将 n 位的二进制整数 u 和 v 各分为 2 段，每段的长为 $n/2$ 位 (为简单起见，假设 n 是 2 的幂)

3.3.2 大整数乘法—分治技术

假定 n 是 2 的幂

$$\begin{array}{l} u = w2^{n/2} + x : \\ v = y2^{n/2} + z : \end{array} \quad \begin{array}{|c|c|} \hline \overbrace{}^{n/2} & \overbrace{}^{n/2} \\ \hline w & x \\ \hline \overbrace{}^{n/2} & \overbrace{}^{n/2} \\ \hline y & z \\ \hline \end{array}$$

u 和 v 的乘积可以计算为：

$$uv = (w2^{n/2} + x)(y2^{n/2} + z) = wy2^n + (xy + wz)2^{n/2} + xz$$

时间复杂度函数为：

$$T(n) = \begin{cases} d & \text{if } n = 1 \\ 4T(n/2) + bn & \text{if } n > 1 \end{cases}$$

递推式的解：

$$T(n) = \Theta(n^2)$$

3.3.2 大整数乘法—分治技术

现在考虑：

$$wz + xy = (w + x)(y + z) - wy - xz$$

我们得到：

$$uv = wy2^n + ((w + x)(y + z) - wy - xz)2^{n/2} + xz$$

时间复杂度是：

$$T(n) = \begin{cases} d & \text{if } n = 1 \\ 3T(n/2) + bn & \text{if } n > 1 \end{cases}$$

递推式的解是：

$$T(n) = \Theta(n^{\log 3}) = O(n^{1.59})$$

3.3.3 矩阵乘法

两个矩阵 A , B : A 的列数等于 B 的行数, 则 A 、 B 可以相乘。即, 如果 $A = (a_{ij})$ 是一个 $m * n$ 的矩阵, $B = (b_{jk})$ 是一个 $n * p$ 的矩阵, 则它们的乘积 $C = AB$ 是 $m * p$ 矩阵 $C = (c_{ik})$ 。

$$\begin{matrix} \begin{bmatrix} 1 & 0 & 2 \\ -1 & 3 & 1 \end{bmatrix} & \times & \begin{bmatrix} 3 & 1 \\ 2 & 1 \\ 1 & 0 \end{bmatrix} & = & \begin{bmatrix} (1 \times 3 + 0 \times 2 + 2 \times 1) & (1 \times 1 + 0 \times 1 + 2 \times 0) \\ (-1 \times 3 + 3 \times 2 + 1 \times 1) & (-1 \times 1 + 3 \times 1 + 1 \times 0) \end{bmatrix} & = & \begin{bmatrix} 5 & 1 \\ 4 & 2 \end{bmatrix} \\ A_{23} & & B_{32} & & & & C_{22} \end{matrix}$$

3.3.3 矩阵乘法—问题描述

? 问题描述

- 令 A 和 B 是两个 $n \times n$ 的矩阵，我们希望计算它们的乘积 $C=AB$ 。

3.3.3 矩阵乘法—传统方法

- 在传统方法中， $C=A \times B$ 是由以下公式计算的

$$C(i, j) = \sum_{k=1}^n A(i, k)B(k, j)$$

- 因为每计算一个元素 $C[i, j]$ ，需要做 n 个乘法和 $n-1$ 次加法
- 很容易看出算法需要 n^3 次乘法运算和 n^3-n^2 次加法运算。这导致其时间复杂度为 $\Theta(n^3)$.

? Is there much more efficient method?

3.3.3 矩阵乘法—递归方法

假定 $n=2^k$, $k \geq 0$, 如果 $n \geq 2$, 则 A , B 和 C 可分别分成 4 个大小为 $n/2 \times n/2$ 的子矩阵

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

用分治方法来计算 C 的组成:

$$C = \begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix}$$

3.3.3 矩阵乘法—递归方法

计算量总耗费是：

$$T(n) = \begin{cases} m & \text{if } n = 1 \\ 8T(n/2) + 4(n/2)^2 a & \text{if } n \geq 2 \end{cases}$$

最终结果：

$$T(n) = (m + \frac{a2^2}{8-2^2})n^3 - (\frac{a2^2}{8-2^2})n^2 = mn^3 + an^3 - an^2$$



观察结论 6.4

- 递归方法需要 n^3 次乘法运算和 $n^3 - n^2$ 次加法运算，这些刚好和前面传统方法所讲述的一样。

两种算法的区别仅仅在于矩阵元素相乘的次序上，其他方面是一样的

3.3.3 矩阵乘法—STRASSEN 算法



基本思想

- 增加加减法的次数来减少乘法次数

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \quad \text{和} \quad B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

$$C = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$



3.3.3 矩阵乘法—STRASSEN 算法



基本思想

我们首先计算以下的乘积

$$d_1 = (a_{11} + a_{22})(b_{11} + b_{22})$$

$$d_2 = (a_{21} + a_{22})b_{11}$$

$$d_3 = a_{11}(b_{12} - b_{22})$$

$$d_4 = a_{22}(b_{21} - b_{11})$$

$$d_5 = (a_{11} + a_{12})b_{22}$$

$$d_6 = (a_{21} - a_{11})(b_{11} + b_{12})$$

$$d_7 = (a_{12} - a_{22})(b_{21} + b_{22})$$

接着从一下面式子计算出 C

$$C = \begin{pmatrix} d_1 + d_4 - d_5 + d_7 & d_3 + d_5 \\ d_2 + d_4 & d_1 + d_3 - d_2 + d_6 \end{pmatrix}$$



3.3.3 矩阵乘法—算法分析

算法用到的加法是 18 次，乘法是 7 次，对于其运行时间产生以下递推式（P121）

$$T(n) = \begin{cases} m & \text{if } n = 1 \\ 7T(n/2) + 18(n/2)^2 a & \text{if } n \geq 2 \end{cases}$$

或

$$T(n) = \begin{cases} m & \text{if } n = 1 \\ 7T(n/2) + (9a/2)n^2 & \text{if } n \geq 2 \end{cases}$$

$$T(n) = \left(m + \frac{(9a/2)2^2}{7 - 2^2}\right)n^{\log_7 7} - \left(\frac{(9a/2)2^2}{7 - 2^2}\right)n^2 = mn^{\log_7 7} + 6an^{\log_7 7} - 6an^2$$

即运行时间为：

$$\Theta(n^{\log_7 7}) = O(n^{2.81})$$

3.3.3 矩阵乘法—算法分析

- Strassen 算法的高效之处，就在于，它成功的减少了乘法次数，将 8 次乘法，减少到 7 次。
- 不要小看这减少的一次，每递归计算一次，效率就可以提高 $1/8$ ，比如一个大的矩阵递归 5 次后， $(7/8)^5 = 0.5129$ ，效率提升一倍。不过，这只是理论值，实际情况中，有隐含开销，并不是最常用算法。
- 矩阵是稀疏矩阵时，为稀疏矩阵设计的方法更快。所以，稠密矩阵上的快速矩阵乘法实现一般采用 Strassen 算法。

3.3.3 矩阵乘法—三个算法的比较

三种算法所做的算术运算的次数

	乘法	加法	复杂度
传统算法.	n^3	$n^3 - n^2$	$\Theta(n^3)$
递归方法.	n^3	$n^3 - n^2$	$\Theta(n^3)$
STRASSEN 算法.	$n^{\log 7}$	$6 n^{\log 7} - 6 n^2$	$\Theta(n^{\log 7})$

3.3.4 最近点对问题—问题描述

? 问题：

- 设 S 是平面上 n 个点的集合，在这一节中，我们考虑在 S 中找到一个点对 p 和 q 的问题，使其相互距离最短。
- 换句话说，希望在 S 中找到具有这样性质的两点 $p_1 = (x_1, y_1)$ 和 $p_2 = (x_2, y_2)$ ，使它们间的距离在所有 S 中点对间为最小。

$$d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

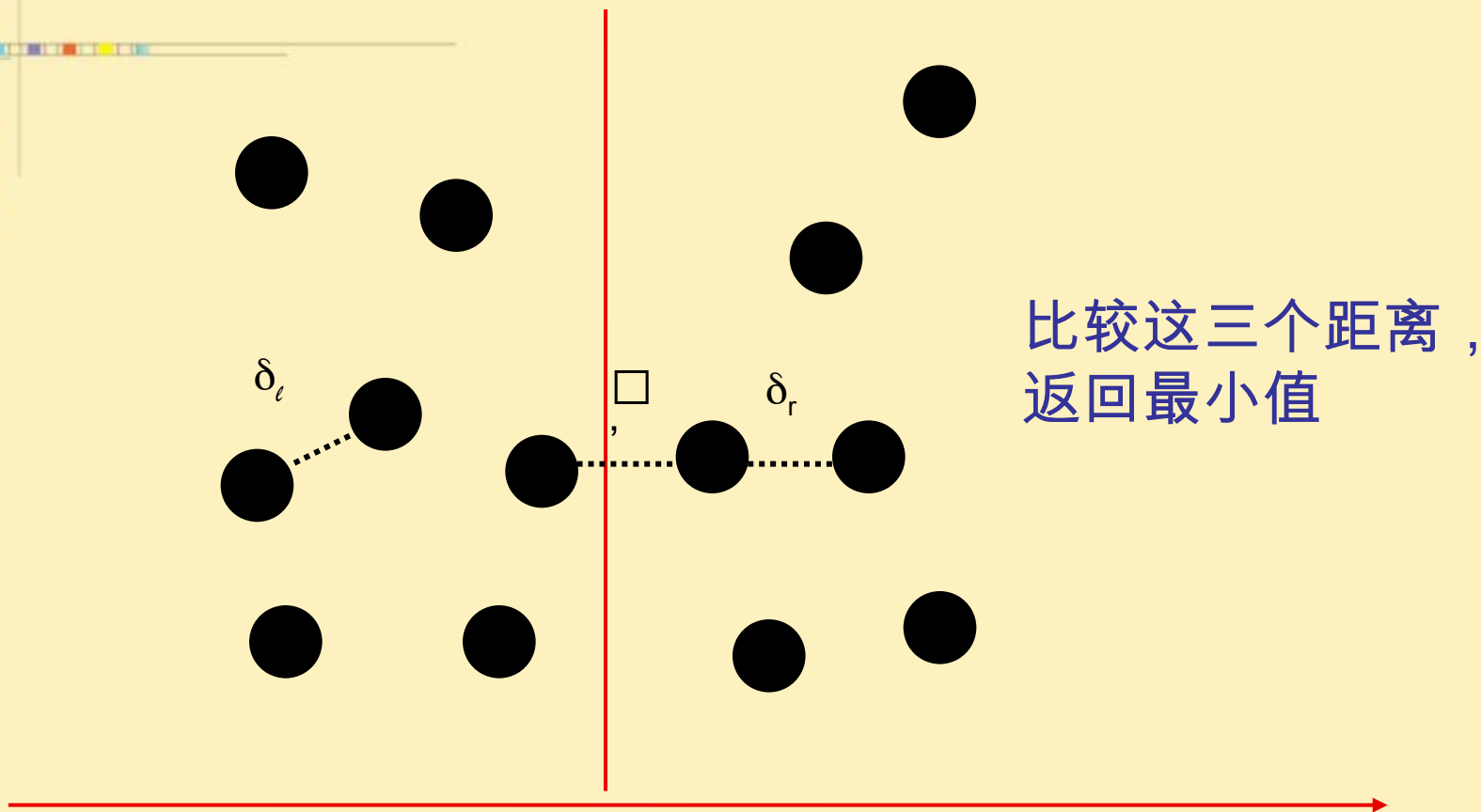
3.3.4 最近点对问题—蛮力算法

- **蛮力算法**就是简单地检验所有可能的 $n(n-1)/2$ 个距离并返回最短间距的点对。耗费 $\Theta(n^2)$ 的时间
- 我们将运用分治设计技术描述一个时间复杂性为 $\Theta(n \log n)$ 的算法来解决最近点对问题。

3.3.4 最近点对问题—分治方法

- Sort:
 - 第一步是以 x 坐标增序对 S 中的点排序
- Divide:
 - S 点集被垂直线 L 大约划分成两个子集 S_l 和 S_r ，使 $|S_l| = \lfloor |S|/2 \rfloor$ ， $|S_r| = \lceil |S|/2 \rceil$ ，设 L 是经过 x 坐标 $S[\lfloor n/2 \rfloor]$ 的垂直线，这样在 S_l 中的所有点都落在或靠近 L 的左边，而所有 S_r 中的点落在或靠近 L 的右边。
- Conquer:
 - 现在按上述递归地进行，两个子集 S_l 和 S_r 的最小间距 $\square_l$ 和 $\square_r$ 可分别计算出来。
- Combine:
 - 组合步骤是把在 S_l 中的点与 S_r 中的点之间的最小间距 $\square'$ 计算出来。最后所要求的解是 $\square_l$ ， $\square_r$ 和 $\square'$ 中的最小值。

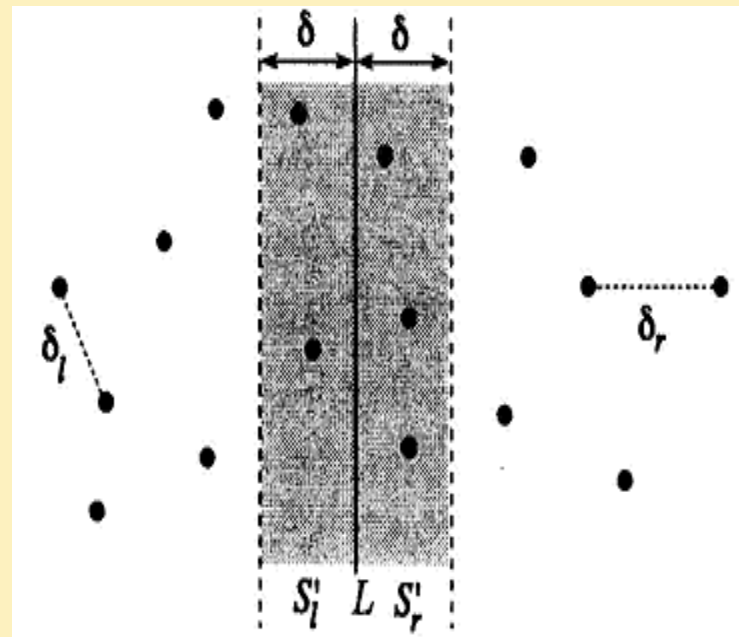
3.3.4 最近点对问题—基本思想



cds

3.3.4 最近点对问题—基本思想

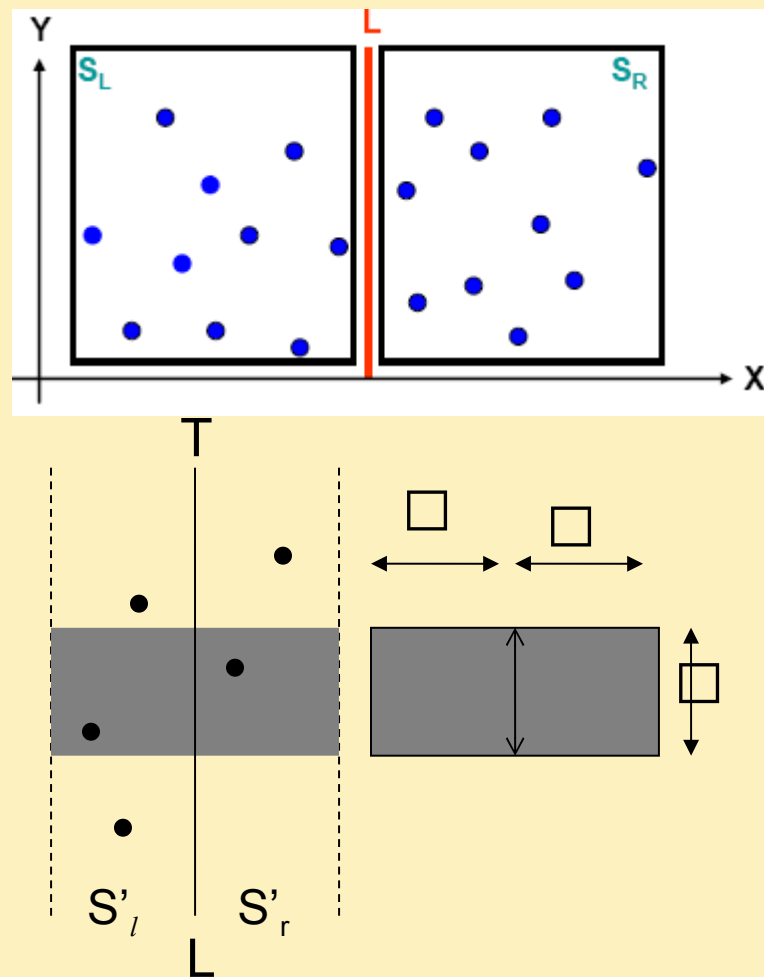
- 怎样合并结果，这一步的关键在于计算 $\square'$ ，计算 S_l 中的每个点与 S_r 中每个点之间的距离的朴素方法需要 $\Omega(n^2)$ 。



- 设 $\square = \min\{\delta_l, \delta_r\}$ ，如果最近点对由 S_l 中的某个点 p_l 与 S_r 中的某个点 p_r 组成，则 p_l 和 p_r 一定在划分线 L 的距离 $\square$ 内。
- 因此，如果令 S_l' 和 S_r' 分别表示为在线 L 距离 $\square$ 内的 S_l 和 S_r 中的点，则 p_l 一定在 S_l' 中， p_r 一定在 S_r' 中。

3.3.4 最近点对问题—基本思想

- 假设 $\square' \leq \square$ ，则存在两点 $p_l \in S'_l$ 和 $p_r \in S'_r$ ，有 $d(p_l, p_r) = \delta'$ ，从而 p_l 和 p_r 之间的垂直距离不超过 $\square$
- 因为 p_l, p_r 这两点都在以垂直线 L 为中心的 $\square \times 2\delta$ 矩形区内或其边界上
- 设 T 是两个垂直带内的点的集合
- 如果在 $\square \times 2\delta$ 矩形区内，任意两点间的距离一定不低于 $\square$ ，则这个矩形最多能容纳 8 个点，其中至多 4 个点属于 S_l ，4 个点属于 S_r 。



3.3.4 最近点对问题—分治方法



观察结论

- T 中的每个点最多需要和 T 中的 7 个点进行比较。
- 上述的观察结论仅给出与 T 中每个点 p 的比较点数的上界，而没有给出哪些点与 P 比较的任何信息。
- 稍想一下，即可知 P 一定是与 T 中临近的点比较。为了找到这样的相邻点，我们借助于以 Y 坐标的递增序对 T 中的点重新排序，然后，不难看出只需对 T 中每个点 P ，与它们的 Y 坐标递增序下的 7 个点比较。



3.3.4 最近点对问题—伪代码

算法 3.3.4 CLOSESTPAIR

输入：平面上 n 个点的集合 S 。

输出： S 中两点的最小距离。

1. 以 x 坐标增序对 S 中的点排序。
2. 序 $Y \leftarrow$ 以 y 坐标增序对 S 中的点排序。
3. $\delta \leftarrow \text{cp}(1, n)$.

过程 $\text{cp}(\text{low}, \text{high})$

1. **if** $\text{high} - \text{low} + 1 \leq 3$ **then** 用直接方法计算 δ
2. **else**
3. $\text{mid} \leftarrow \lfloor (\text{low} + \text{high}) / 2 \rfloor$
4. $x_0 \leftarrow x(S[\text{mid}])$
5. $\delta_l \leftarrow \text{cp}(\text{low}, \text{mid})$
6. $\delta_r \leftarrow \text{cp}(\text{mid} + 1, \text{high})$
7. $\delta \leftarrow \min\{\delta_l, \delta_r\}$
8. $k \leftarrow 0$

To be continued...

3.3.4 最近点对问题—伪代码

过程 $cp(low, high)$

```
9.      for  $i \leftarrow 1$  to  $n$            { 从  $Y$  中抽取  $T$  }
10.      if  $|x(Y[i]) - x_0| \leq \delta$  then
11.           $k \leftarrow k + 1$ 
12.           $T[k] \leftarrow Y[i]$ 
13.      end if
14.  end for
15.   $\delta' \leftarrow 2\delta$ 
16.  for  $i \leftarrow 1$  to  $k - 1$ 
17.      for  $j \leftarrow i + 1$  to  $\min\{i + 7, k\}$ 
18.          if  $d(T[i], T[j]) < \delta'$  then  $\delta' \leftarrow d(T[i], T[j])$ 
19.      end for
20.  end for
21.   $\delta \leftarrow \min\{\delta, \delta'\}$ 
22. end if
23. return  $\delta$ 
```

3.3.4 最近点对问题—算法分析

- 排序 S 中的点需要 $\Theta(n \log n)$ 时间。
- 把点划分成 S_l 和 S_r 子集需要 $D(1)$ 时间，因为此时点已排序了。
- 至于组合步骤，我们可以看到它包含对 T 中点的排序和每点与其他至多 7 个点做比较，排序耗费 $O(n \log n)$ 时间，并存在至多 $7n$ 次比较，这样组合步骤最坏情况下需要 $\Theta(n \log n)$ 时间。
- 注意如果点数为 3，则计算它们间最小间距只需要做三次比较，算法执行的递推关系变为：

$$T(n) = \begin{cases} 1 & \text{if } n = 2 \\ 3 & \text{if } n = 3 \\ 2T(n/2) + O(n \log n) & \text{if } n > 3 \end{cases} \quad \rightarrow T(n) = O(n \log^2 n)$$

3.3.4 最近点对问题—算法分析

在组合步骤中，每次需要对 T 排序，现在只需要在 $\Theta(n)$ 时间里从数组 Y 中提取它的元素，这样递推式变为：

$$T(n) = \begin{cases} 1 & \text{if } n = 2 \\ 3 & \text{if } n = 3 \\ 2T(n/2) + \Theta(n) & \text{if } n > 3 \end{cases} \quad \rightarrow T(n) = O(n \log n)$$

3.3.4 最近点对问题—时间复杂度

- 假定有平面上 n 个点的集合 S ，算法 CLOSESTPAIR 在 $O(n \log n)$ 时间内找到 S 中具有最小间距的一个点对。

3.3.6 多数元素问题—问题描述

- 给定一个大小为 n 的数组，找到其中的多数元素。多数元素是指在数组中出现次数大于 $\lfloor n/2 \rfloor$ 的元素。
- 你可以假设数组是非空的，并且给定的数组总是存在多数元素。
- 首先就是要将给定的数组分成两部分【分解】，分别求解这两部分的众数【求解】，然后比较这两部分的众数，最终得出结果【合并】

3.3.6 多数元素问题—基本思想

- 根据多数元素的定义可知多数元素必定在数组中出现次数最多，而若将原数组分成2部分，则在其中一部分出现次数最多的必定是多数元素，证明如下：
 - 假定数组长度为 n ，多数元素为 a ，出现次数为 $f_n(a)$ ，已知： $f_n(a) > n/2$
 - 将数组分成两部分，长度分别为 l ， r ，其中 $l+r = n$
 - 若假设 $f_l(a) \leq l/2$ ， $f_r(a) \leq r/2$ 则 $f_n(a) \leq n/2$ ，与已知条件相矛盾，故假设不成立
 - 故将原数组分成2部分，则在其中一部分出现次数最多的必定是多数元素

3.3.6 多数元素问题 - 算法实现

```
1. int majorityElementRec (int[] nums, int low, int high) {  
2.     if (low == high) {  
3.         return nums[low];  
4.     }  
5.     int mid = (high + low) / 2 ;  
6.     int left = majorityElementRec (nums, low, mid);  
7.     int right = majorityElementRec (nums, mid + 1, high);  
8.     if (left == right) {  
9.         return left;  
10.    }  
11.    int leftCount = countInRange(nums, left, low, high);  
12.    int rightCount = countInRange(nums, right, low, high);  
13.    return leftCount > rightCount ? left : right;  
14. }  
15. }
```

3.3.6 多数元素问题 - 算法实现

- `int countInRange(int[] nums, int num, int low, int high)`
- `{`
- `int count = 0;`
- `for (int i = low; i <= high; i++) {`
- `if (nums[i] == num) {`
- `count ++;`
- `}`
- `}`
- `return count;`
- `}`

3.3.6 多数元素问题—算法分析

- 时间复杂度: $O(n \log n)$
- 空间复杂度: $O(n \log n)$



3.3.7 循环赛日程表—问题描述

? 问题描述:

设有 n 个运动员要进行网球循环赛。现要设计一个满足以下要求的比赛日程表:

设计一个满足以下要求的比赛日程表:

- (1) 每个选手必须与其他 $n-1$ 个选手各赛一次;
- (2) 每个选手一天只能赛一次;
- (3) 循环赛一共进行 $n-1$ 天。

3.3.7 循环赛日程表—基本思想

基本思想：

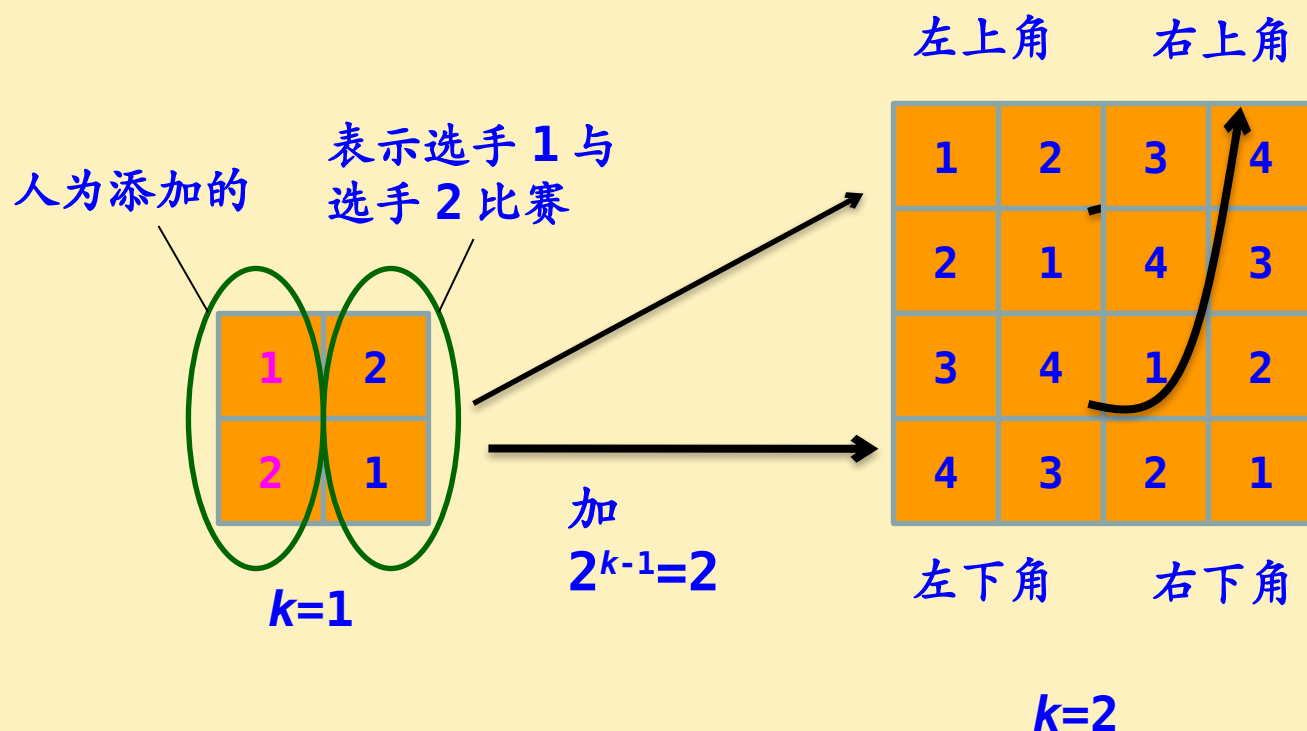
- 按问题要求可将比赛日程表设计成一个 n 行 $n-1$ 列的二维表，其中第 i 行、第 j 列表示和第 i 个选手在第 j 天比赛的选手。

假设 n 位选手被顺序编号为 $1、2、\dots、n$ (2^k)。



3.3.7 循环赛日程表—基本思想

由 $k=1$ 创建 $k=2$ 的过程



3.3.7 循环赛日程表—基本思想

由 $k=2$ 创建 $k=3$ 的过程

1	2	3	4
2	1	4	3
3	4	1	2
4	3	2	1

$k=2$

加

$2^{k-1}=4$

左上角				右上角			
1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1
左下角				右下角			

$k=3$

3.3.7 循环赛日程表—基本思想

- 按此要求将比赛日程表设计成有 n 行和 $n-1$ 列的一个表。在表中的第 i 行，第 j 列处填入第 i 个选手在第 j 天所遇到的选手。

选手	第 1 天	第 2 天	第 3 天	第 4 天	第 5 天	第 6 天	第 7 天
1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

3.3.7 循环赛日程表—基本思想

将 $n=2^k$ 问题划分为 4 部分:

(1) **左上角**: 左上角为 2^{k-1} 个选手在前半程的比赛日程 ($k=1$ 时直接给出, 否则, 上一轮求出的就是 2^{k-1} 个选手的比赛日程)。

(2) **左下角**: 左下角为另 2^{k-1} 个选手在前半程的比赛日程, 由左上角加 2^{k-1} 得到, 例如 2^2 个选手比赛, 左下角由左上角直接加 2

(2^{k-1}) 得到, 2^3 个选手比赛, 左下角由左上角直接加 4 (2^{k-1}) 得到。

(3) **右上角**: 将左下角直接复制到右上角得到另 2^{k-1} 个选手在后半程的比赛日程。

(4) **右下角**: 将左上角直接复制到右下角得到 2^{k-1} 个选手在后半程的比赛日程。

3.3.7 循环赛日程表 - 算法实现

```
#include <stdio.h>
#define MAX 101
// 问题表示
int k; // 求解结果表示
int a[MAX][MAX]; // 存放比赛日程表 (行列下标为 0 的元素不用)

void Plan(int k)
{
    int i, j, n, t, temp;
    n=2; // n 从  $2^1=2$  开始
    a[1][1]=1; a[1][2]=2; // 求解 2 个选手比赛日程, 得到左上角元素
    a[2][1]=2; a[2][2]=1;

    for (t=1; t<k; t++) // 迭代处理  $2^2(t=1) \dots, 2^k(t=k-1)$  个选手
    {
        temp=n; // temp= $2^t$ 
        n=n*2; // n= $2^{(t+1)}$ 

        for (i=temp+1; i<=n; i++) // 填左下角元素
            for (j=1; j<=temp; j++)
                a[i][j]=a[i-temp][j]+temp; // 产生左下角元素

        for (i=1; i<=temp; i++) // 填右上角元素
            for (j=temp+1; j<=n; j++)
                a[i][j]=a[i+temp][(j+temp)% n];

        for (i=temp+1; i<=n; i++) // 填右下角元素
            for (j=temp+1; j<=n; j++)
                a[i][j]=a[i-temp][j-temp];
    }
}
```